Trustworthy AI for Open RAN — Analysis Report

Audience: Non-specialist

Generated:

Run on cci_a1sd_binary with Xgboost: Accuracy 100.0%

Personal research project — not affiliated with any employer.

Contents

- In one page: does it work, and can you trust it
- What it was tested on, and what was built
- How often it gets it right
- What the model looks at (explainability)
- What happens if someone fools it (robustness)
- Whether it plays favourites (fairness)
- Whether it could give your data away (privacy)
- What the simulated control room did
- The agent's check-up scores
- Where these numbers come from

In one page: does it work, and can you trust it

This page answers the questions anybody asks about a system like this, in plain words. Every number in it is a measurement taken by the run described later in this report — nothing here is estimated separately.

Does it actually work?

Tested only on the part of the data the model had never seen, it answered correctly 100.0% of the time. That is a good result.

Judged kind by kind — so that a rare kind cannot be hidden by a common one — the score is 100.0%. Nothing to act on here.

Can you see why it decided that?

When the model decides, these are the measurements it leans on most: inter-arrival jitter: the smaller it is, the keener the model is on answering "DoS"; mean response time: the smaller it is, the keener the model is on answering "DoS"; payload-size jitter: the smaller it is, the keener the model is on answering "DoS"; mean payload size: the larger it is, the keener the model is on answering "DoS"; inter-arrival CV: the smaller it is, the keener the model is on answering "DoS".

None of these are hidden internals: every lever on the answer is a measurement an operator can look up and argue about.

What if someone tries to fool it?

Untampered, the model is right 100.0% of the time. Then we let a program tamper with the input the way an attacker would, three ways of increasing cleverness — one quick nudge: 47.3%; a patient attack that keeps probing: 68.0%; the smallest change that still works: 53.3%.

Of the three, the one that bites hardest takes the answer from 100.0% down to 47.3%, a loss of 52.7 percentage points — a cost we measured in advance rather than discovered afterwards. This one needs attention.

Does it treat every group alike?

The samples split into 8 groups by Policy name. The question is whether every group gets the same accuracy and the same willingness to raise an alert, instead of one quietly getting a laxer or a harsher treatment.

Could it give your data away?

The same test rerun through 4 shadow models that the outsider trains themselves gives 0.51 (group attack) and 0.4904 (likelihood-ratio attack), so the reading does not rest on a single attack design.

The last section of this report lists what these numbers do not prove — worth a read before quoting any of them.

What it was tested on, and what was built

The data describes A1 traffic: Normal vs DoS.

It holds 24000 samples, 10 measurements each, all taken from one event.

The model's job is to sort each sample into one of 2 kinds: Normal; DoS.

What was built: Xgboost — that is, a tuned chain of correctors (XGBoost). Putting it together and running every check took 5.95 seconds.

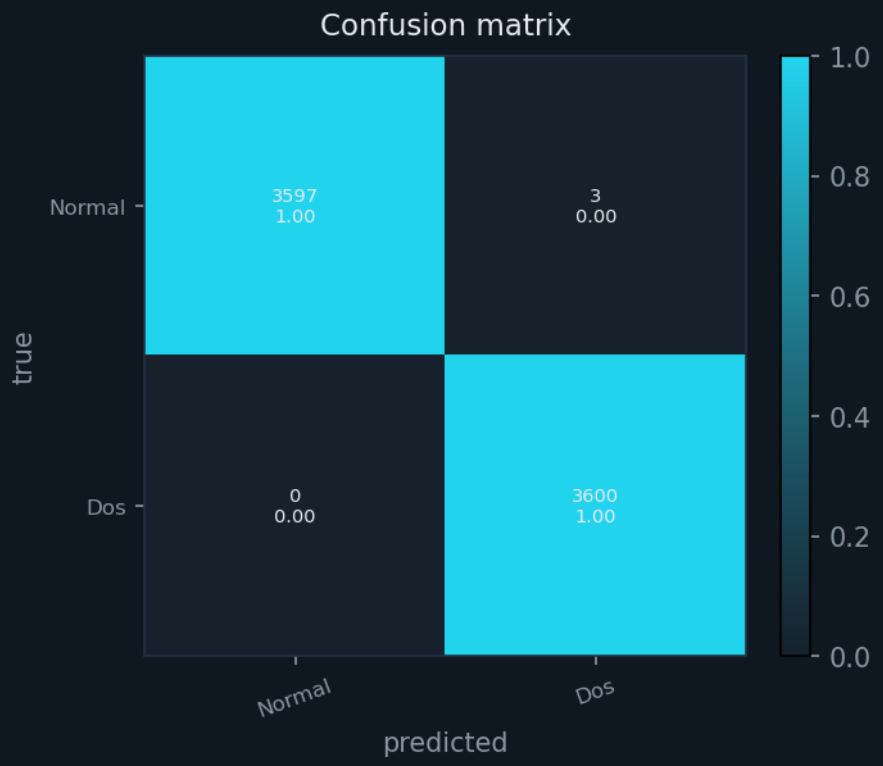
Every measurement by name

actual inter-arrival; mean inter-arrival; inter-arrival jitter; inter-arrival CV; mean response time; response-time jitter; request rate; mean payload size; payload-size jitter; request body size.

How often it gets it right

Tested only on the part of the data the model had never seen, it answered correctly 100.0% of the time. That is a good result.

Judged kind by kind — so that a rare kind cannot be hidden by a common one — the score is 100.0%. Nothing to act on here.



Which kinds get mixed up

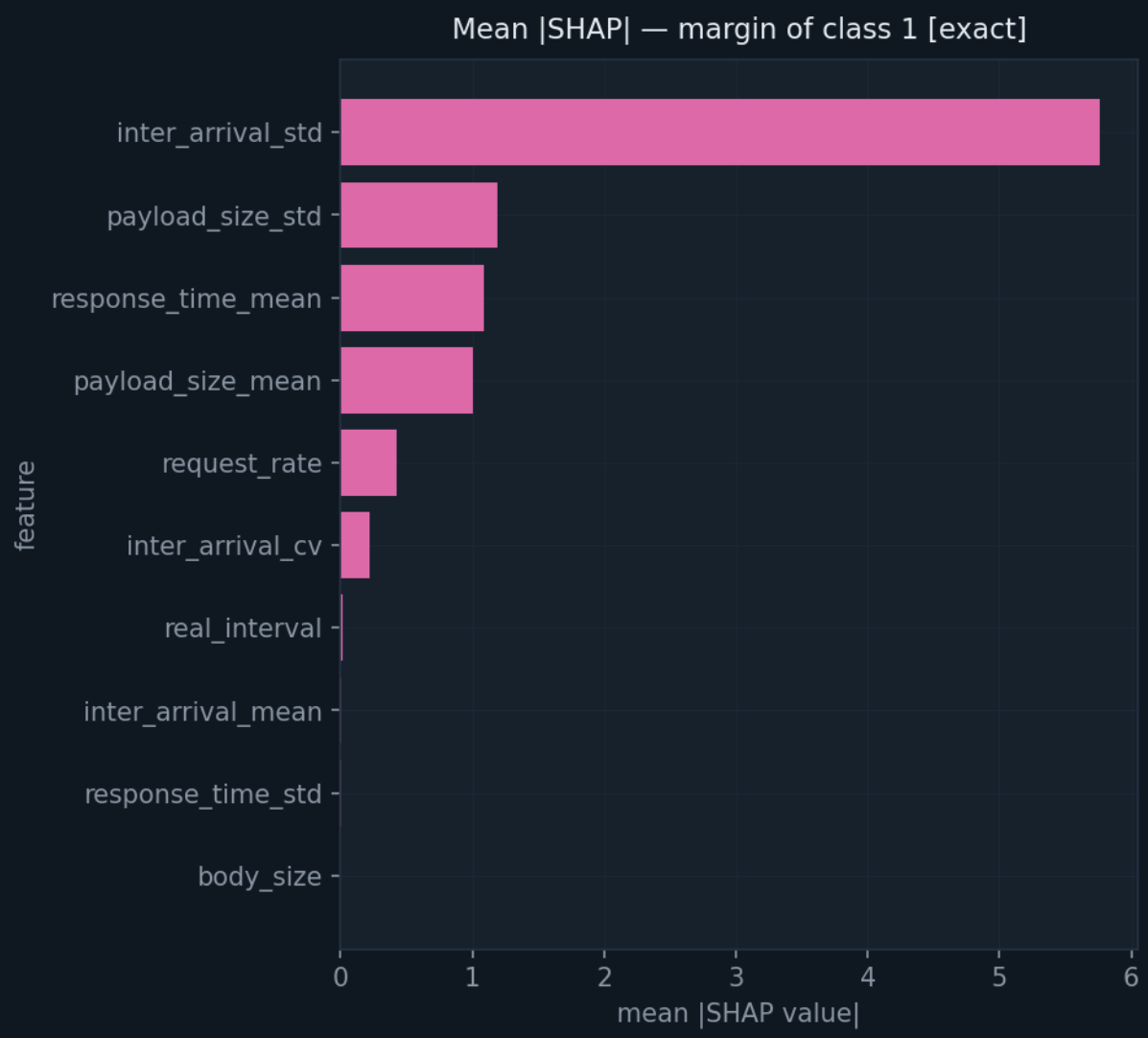
What the model looks at (explainability)

This part answers one thing: what the model was actually looking at, so the answer can be argued about instead of taken on faith.

When the model decides, these are the measurements it leans on most: inter-arrival jitter: the smaller it is, the keener the model is on answering “DoS”; mean response time: the smaller it is, the keener the model is on answering “DoS”; payload-size jitter: the smaller it is, the keener the model is on answering “DoS”; mean payload size: the larger it is, the keener the model is on answering “DoS”; inter-arrival CV: the smaller it is, the keener the model is on answering “DoS”.

None of these are hidden internals: every lever on the answer is a measurement an operator can look up and argue about.

One single case, taken apart — what pushed this one towards “DoS”: inter-arrival jitter (+0.812); payload-size jitter (+0.061); what pulled it back: request rate (-0.070); response-time jitter (-0.029).



How much each measurement moves the answer

What happens if someone fools it (robustness)

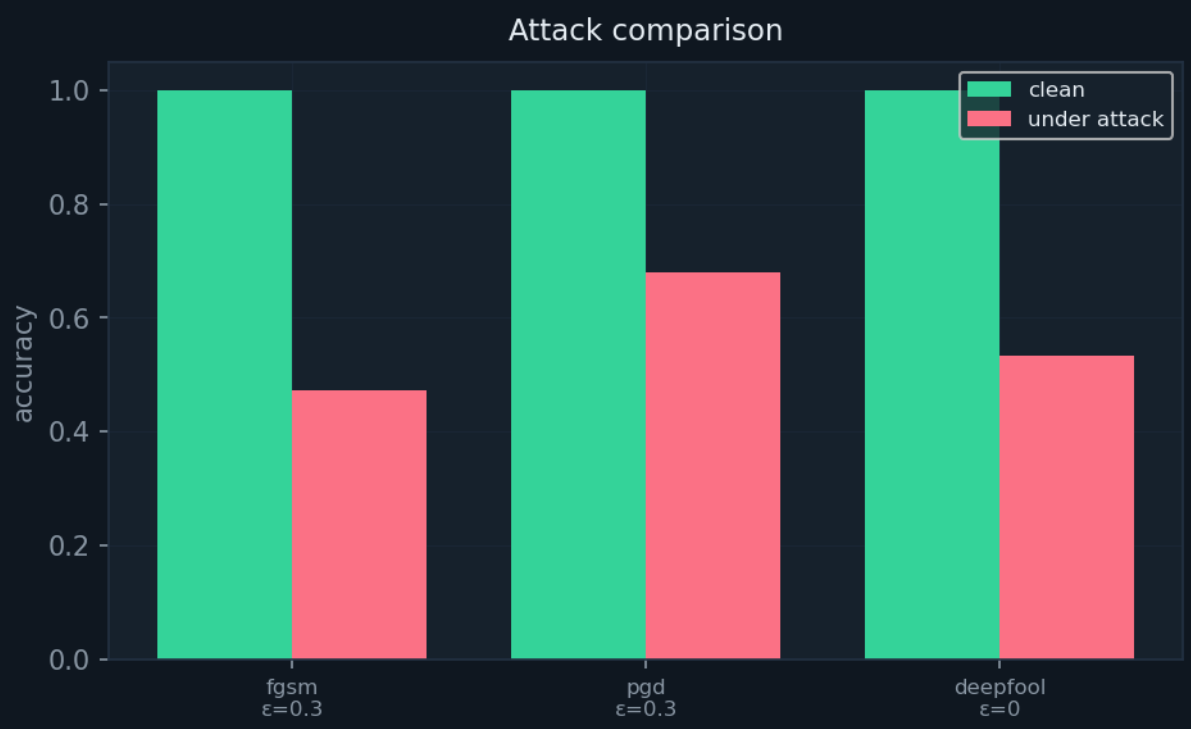
Here we tamper with the input on purpose, to see how fast the answers stop being right.

Untampered, the model is right 100.0% of the time. Then we let a program tamper with the input the way an attacker would, three ways of increasing cleverness — one quick nudge: 47.3%; a patient attack that keeps probing: 68.0%; the smallest change that still works: 53.3%.

Of the three, the one that bites hardest takes the answer from 100.0% down to 47.3%, a loss of 52.7 percentage points — a cost we measured in advance rather than discovered afterwards. This one needs attention.

Counted one by one, the probing attack changed the model's mind on 32.0% of the samples it was given. This is the weak spot: do not lean on the answer until it is fixed.

There is a remedy: retraining the same model on tampered data brings it back to 96.7%, 49.3 points better than the untoughened model under the same attack.



Accuracy under three kinds of tampering

Whether it plays favourites (fairness)

This part asks whether one group of sites, agents or platforms quietly gets better treatment than another.

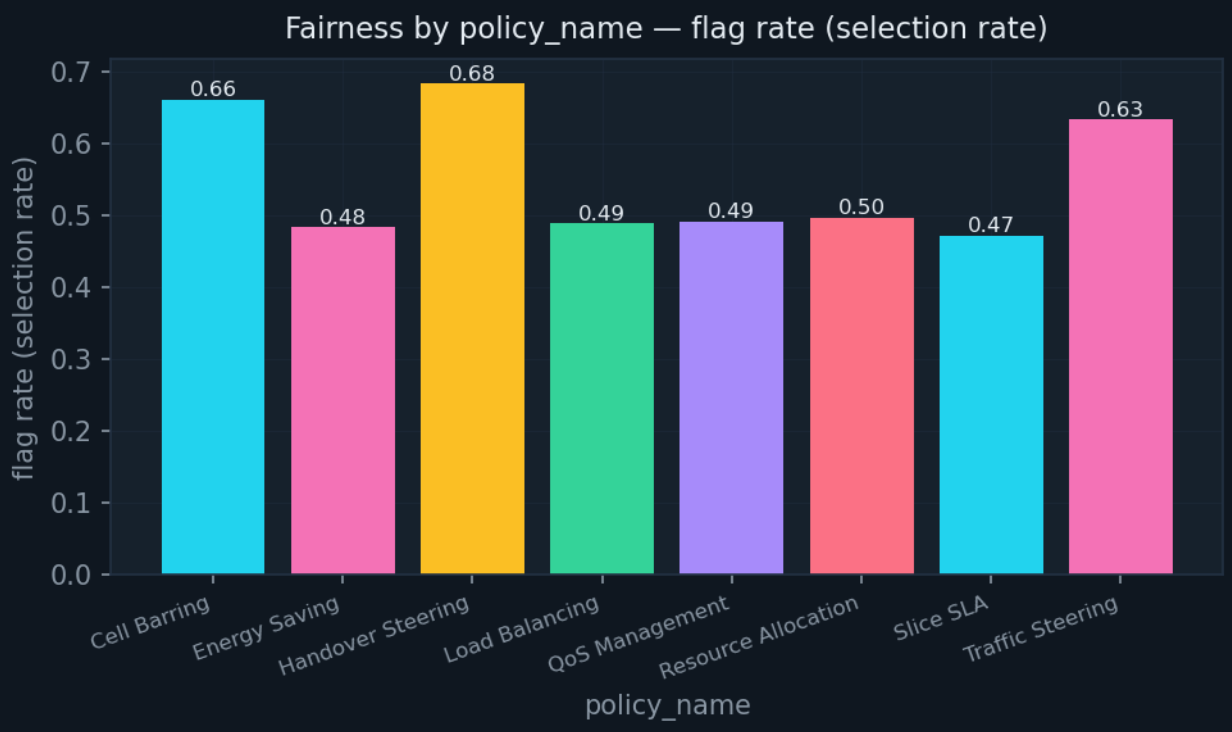
The samples split into 8 groups by Policy name. The question is whether every group gets the same accuracy and the same willingness to raise an alert, instead of one quietly getting a laxer or a harsher treatment.

Group by group, the best treated is Cell Barring (right 100.0% of the time) and the worst is Traffic Steering (99.6%) — 0.4 percentage points apart. That is a good result.

As for how often an alert is raised: the most alerted group is Handover Steering (68.3% of its samples), the least is Slice SLA (47.1%).

There is a common rule of thumb — the least alerted group should still reach at least four fifths (0.8) of the most alerted one. This run measures 0.69, so it falls short, and the policy mix behind it is worth rebalancing.

Among the samples that genuinely deserved an alert, the largest difference in "was it actually caught" between groups is 0.00 — 0 would mean every group is protected equally. Nothing to act on here.



Group by group comparison

Whether it could give your data away (privacy)

And here: does training the model hand over anybody's individual measurements?

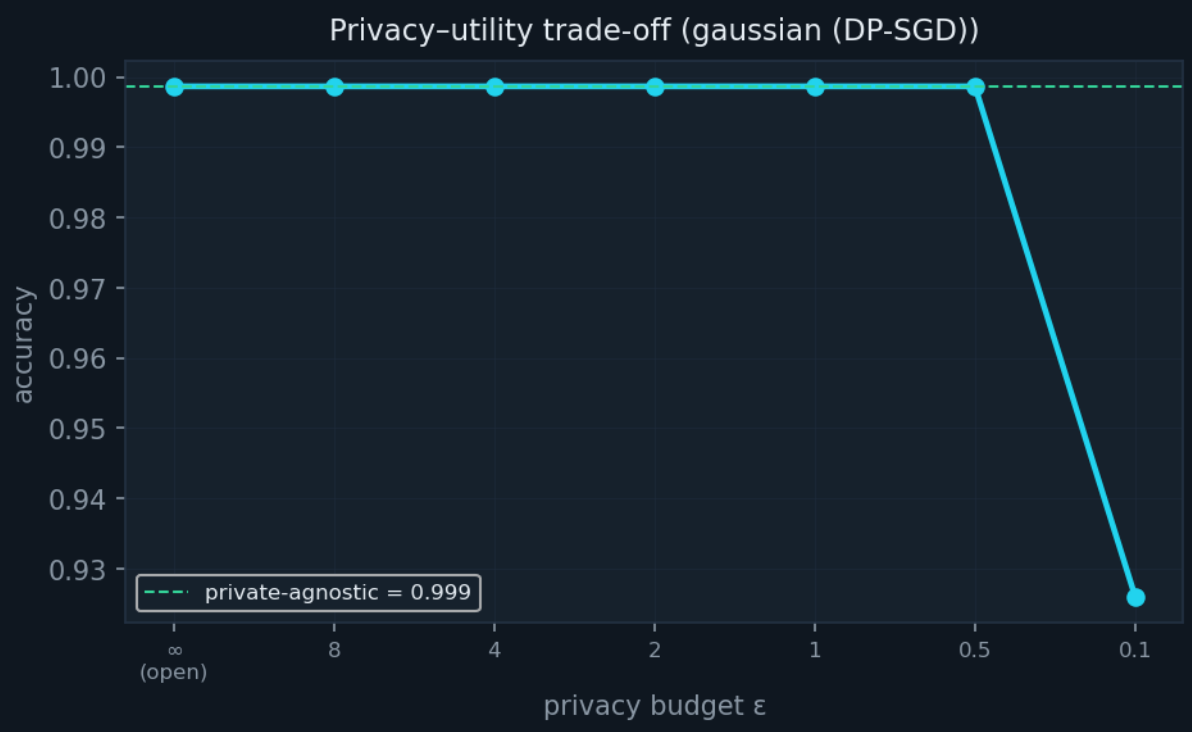
The shield is noise mixed into training so that no single record can be picked out afterwards, and ε is its dial: the smaller ε, the stronger the shield and the more model utility it costs.

Measured on that dial — no protection at all: 99.9%; ε=1: 99.9%; ε=0.1: 92.6%.

In other words the shield costs about 0.0 percentage points of accuracy (and a further 7.3 to reach the stricter setting). That is a good result.

We also tested whether the released model gives away its training records: an outsider holding a copy of the model and a record with its true label guesses whether that record was used in training. Pure guessing scores 0.5; this run scores 0.49, which is close enough to guessing that no individual record can be picked out. Nothing to act on here.

The same test rerun through 4 shadow models that the outsider trains themselves gives 0.51 (group attack) and 0.4904 (likelihood-ratio attack), so the reading does not rest on a single attack design.



Privacy protection traded against model utility

What the simulated control room did

This is the studio's own control-room simulation — a model of the loop, not a live network.

Nothing was captured: no simulation session was running when this report was written.

The agent's check-up scores

No agent evaluation has been run yet — run it in the Agent Eval panel to fill this section.

Where these numbers come from

How these numbers were produced, in one paragraph: the system learned from part of the data and was then tested only on the part it had never seen, so the score is not a memory test. The random seed is fixed, which means running the whole check again gives the same figures instead of a fresh roll of the dice.

Everything in this report came out of that single check, which is why the numbers in the text and the numbers in the charts agree.

The same check can be re-run from the command line with python -m trustoran --dataset cci_a1sd_binary --quick.

What these numbers do not prove

- The data is published research material, not traffic taken off a real network, so the scores describe this exercise rather than any live site.
- The tampering we tried is three well-known ways of attacking a model like this. An attacker with a different trick could behave differently.
- The privacy figures are our own measurement of how much accuracy the noise costs. They are not a legal certificate that no data can ever be recovered.
- Which groups we compared is a choice we made, not a fact about the data. Compare a different field and the fairness numbers will move.

- The agent was tested inside a sealed practice box. Nothing it did reached a real computer, account or network.

This module is a personal research project that extends trustworthy-AI methodology to agent evaluation; it is not affiliated with any employer. The O-RAN scenarios and standards summaries are the author's own paraphrasing of public material.